MediDispense: complete setup and user guide

For students. Follow it in order, one step at a time. It assumes you have never used a terminal before, so nothing is skipped.

How to use this guide

- Do the parts **in order**. Do not skip ahead, and do not do two steps at once.
- A **dark box** is something you **type or paste** into PowerShell exactly as written, then press **Enter**. Wait until it finishes before the next step.
- A **light grey box** after a command shows what you **should see**. A green box means success; a yellow box is a warning; a blue box is a tip.
- If what you see is different, go to **Part 14 (If something goes wrong)** before carrying on.
- These steps are for **Windows 10 or 11**. You need a laptop where you can install programs, and an internet connection for the first setup.
- Time: about 1 hour for the software and 2 to 3 hours to build the conveyor.

What you are building. A doctor's prescription creates a delivery job. A small ESP32-driven conveyor carries a medicine box to Bed 1 or Bed 2, a servo pushes it into that bed's chute, and a nurse confirms it at the bedside. Everything is stored in a database and shown live in a web app. There are **four downloads** (all under **github.com/SHIVAAKARTHIK**): `conveyor-backend` (the server and database), `conveyor-admin-web` (the web app), `conveyor-firmware-` (the program for the ESP32 board; note the hyphen at the end), and `conveyor-docs` (this guide).

1. What you need
2. Terminal basics (read this first)
3. Install the software
4. Download the project
5. Start the backend
6. Start the web app
7. First login and accounts
8. Build the conveyor
9. Put the firmware on the board
10. Test the hardware
11. Create and run a delivery job
12. Every time you use it
13. WiFi, new computer, updates, reset
14. If something goes wrong
15. Before real use, quick reference

1. What you need

Hardware

Qty	Part	Notes
1	ESP32 DevKit V1 (30-pin)	The board that runs the conveyor
1	USB cable (USB-A to Micro-USB)	It must carry **data**, not only charge. If the board never appears as a COM port, try another cable.
1	L298N motor driver	Switches the belt motor
1	Conveyor belt kit with DC gear motor	About 20 cm belt, 6.5 cm wide (we used the "Move It!" kit)
2	SG90 micro servo	One pusher per bed
2	IR obstacle sensor module	One per bed; it detects the box
1	Active buzzer module	The beeper

1	Push button (tactile)	Starts a delivery
1	12 V, 2 A DC adapter	Powers the motor driver only
1 set	Jumper wires (male-male, male-female, female-female)	About 40 pieces
1	A small test box	About 4 x 3 x 1.5 cm (a matchbox-size box, empty or lightly filled)

Tools and materials: a soldering iron and solder, a small screwdriver (for the motor-driver terminals), thin plastic sheet and cardboard (for the pusher arms, servo risers and the chutes), glue and tape, scissors or a craft blade, and a ruler. A multimeter helps but is not required.

Software and accounts

Item	Used for
Git	Downloads the project
Python 3.13	Runs the backend
Node.js (version 22 or newer)	Runs the web app
arduino-cli	Builds and flashes the ESP32 (not needed if you install the firmware from the browser)
Chrome or Edge	Opens the web app; the browser installer needs one of these
A phone or second laptop	Used once, to enter the conveyor's WiFi details (Part 9)

All the software is **free**. You do **not** need a GitHub account. The computer and the conveyor must be on the **same 2.4 GHz WiFi**. College WiFi that blocks devices from talking to each other (often called guest, client isolation or AP isolation) will not work: use a phone hotspot or a home router, and set the hotspot to **2.4 GHz**.

2. Terminal basics (read this first)

Most steps use **PowerShell**, a window where you type commands. It is already on every Windows computer.

2.1 Open PowerShell

1. Press the **Windows key**.
2. Type `PowerShell`.
3. Press **Enter**. A blue (or black) window opens. The last line, called the **prompt**, looks like `PS C:\Users\yourname>`. That means it is waiting for you to type.

2.2 Open PowerShell inside a folder (the easy way)

Many steps say "open PowerShell in the folder". Do this:

1. Open the folder in **File Explorer** (the yellow folder icon).
2. Click once in the **address bar** at the top, the box that shows the folder path.
3. Type `powershell` and press **Enter**.
4. A PowerShell window opens, and its prompt now ends with that folder's name. That is the folder every command will run in.

The other way is to type `cd` and the folder path, for example `cd C:\MediDispense\conveyor-backend`, then Enter. The folder names are written with a backslash `\`.

2.3 Typing and pasting commands

- Type the command exactly as shown, or copy it from this guide. **Do not type the prompt** (`PS C:\...>`) itself.
- To paste in PowerShell, **right-click** in the window, or press **Ctrl+V**. Then press **Enter**.
- A long command may wrap onto the next line in this guide or in the window. It is still **one command**: do not press Enter in the middle.
- Capital letters, dots, hyphens and spaces all matter. `.venv` starts with a dot. `conveyor-firmware-` ends with a hyphen.
- When a command finishes, a new prompt appears. If there is no new prompt, it is still working: wait.
- **Copying from this PDF:** if a pasted command breaks into two lines and gives an error, paste it into **Notepad** first, join it into one line, then copy it again into PowerShell. All the commands in this guide are written to fit on one line.

2.4 Moving around and looking

Type	What it does
`dir`	Lists the files and folders in the current folder
`cd C:\MediDispense`	Goes to that folder
`cd ..`	Goes up one folder
Tab key	Completes a name: type `cd conveyor-b` and press **Tab**
Up arrow	Brings back the previous command

2.5 Stopping a program, and windows you must keep open

The backend and the web app run **continuously**. While one is running, its PowerShell window does not show a new prompt: **that is normal**. Leave the window open. To stop it, click in that window and press **Ctrl+C** (or close the window). Because each one occupies a window, you will open a **separate** PowerShell window for each.

2.6 Copying a file (for example config.h.example to config.h)

Some steps need you to **copy a template file and give the copy a new name**. The original stays unchanged. There are two ways; use either.

Way 1: with a command (recommended)

Open PowerShell **in the folder that holds the file** (section 2.2) and type `copy` , the old name, and the new name:

```
copy config.h.example config.h
```

No message means it worked. Check it with `dir` : you now see both `config.h.example` and `config.h` .

Way 2: with File Explorer (only for config.h)

1. First make file endings visible. **Windows 11:** in File Explorer click **View**, then **Show**, then tick **File name extensions**. **Windows 10:** click the **View** tab and tick **File name extensions**.
2. Open the folder. Right-click `config.h.example` and choose **Copy** (Windows 11: the copy icon).
3. Right-click an empty space in the folder and choose **Paste**. A new file appears, named `config.h - Copy.example` or similar.
4. Right-click that new file, choose **Rename**, type exactly `config.h` , and press **Enter**.
5. If Windows warns that changing the extension may make the file unusable, click **Yes**.

> The files named `.env` and `.env.local` start with a dot, and Windows Explorer will not let you rename a file to that. **Always use the command** for those two.

2.7 Run PowerShell as administrator (needed once, in Part 5)

Press the **Windows key**, type `PowerShell`, then **right-click** "Windows PowerShell" and choose **Run as administrator**. Click **Yes** when Windows asks. The window title then starts with "Administrator".

3. Install the software

> **Important:** after installing any program below, **close every PowerShell window and open a new one**. An already-open window does not know about newly installed programs, and shows "is not recognized" errors.

There are two ways for each program. **Way A (fastest)** is one line using `winget`, which is built into Windows 10 and 11. If it says `winget` is not recognized, or you prefer clicking, use **Way B** (download and run the installer, accepting the default options unless this guide says otherwise).

3.1 Git

Way A: `winget install -e --id Git.Git` **Way B:** download from **git-scm.com** (Download for Windows), run it and keep clicking Next, then Install.

Close PowerShell, open a new one, and check:

```
git --version
```

```
git version 2.xx.x.windows.x
```

3.2 Python 3.13

Way A: `winget install -e --id Python.Python.3.13` **Way B:** download from **python.org/downloads**. On the **first screen of the installer, tick "Add python.exe to PATH"** (a small box at the bottom), then click **Install Now**.

Close PowerShell, open a new one, and check:

```
python --version
```

```
Python 3.13.x
```

> If it says "*Python was not found; run without arguments to install from the Microsoft Store*", Windows is showing a shortcut instead of the real Python. Either reinstall Python from python.org with "Add python.exe to PATH" ticked, or switch the shortcut off: **Settings, Apps, Advanced app settings, App execution aliases**, then turn **off** "python.exe" and "python3.exe". Open a new PowerShell and try again.

3.3 Node.js

Way A: `winget install -e --id OpenJS.NodeJS.LTS` **Way B:** download the **LTS** installer from **nodejs.org** and run it with the default options. The project was tested on version 22; version 22 or newer is required.

Close PowerShell, open a new one, and check both:

```
node --version
npm --version
```

```
v22.x.x
10.x.x
```

3.4 arduino-cli

Way A: `winget install -e --id ArduinoSA.CLI`

Way B: go to **arduino.github.io/arduino-cli**, open Installation, and download the **Windows 64-bit** zip. Unzip it, create a folder `C:\arduino-cli`, and move `arduino-cli.exe` into it. Then add that folder to PATH: press the Windows key, type `environment variables`, open **Edit environment variables for your account**, select **Path**, click **Edit**, **New**, type `C:\arduino-cli`, and click OK on every window.

Close PowerShell, open a new one, and check:

```
arduino-cli version
```

```
arduino-cli  Version: 1.x.x ...
```

If you will only install the firmware from the browser (Part 9, option B), you can skip this one.

3.5 Chrome or Edge

Use Google Chrome or Microsoft Edge on a computer for the web app. (Firefox and Safari cannot install the firmware from the browser.)

3.6 The USB driver for the board

Plug the ESP32 into the computer with the USB cable. Windows usually installs the driver by itself. Check:

1. Right-click the **Start** button and choose **Device Manager**.
2. Click the small arrow next to **Ports (COM & LPT)**.
3. You should see **Silicon Labs CP210x USB to UART Bridge (COM3)**. The number may be different (COM4, COM5...). **Write down your number.**

> If there is no "Ports" entry, or the board shows as **Unknown device** or **USB Serial** with a yellow mark, the driver is missing. Search the web for **"CP210x Universal Windows Driver"** from Silicon Labs, download and unzip it, right-click `silabser.inf` and choose **Install**. (A board with a **CH340** chip needs the "CH341SER" driver instead.) Then unplug and replug the board. If it still does not appear, try a different USB cable, because many cables only charge.

4. Download the project

4.1 Make a folder for everything

Open a new PowerShell (section 2.1) and type:

```
mkdir C:\MediDispense
cd C:\MediDispense
```

The prompt now ends with `C:\MediDispense>`. Use this folder, not Desktop, Documents or OneDrive: those can cause problems, and names with spaces make commands harder.

4.2 Download the four repositories

Type these **one at a time**, pressing Enter after each, and wait for each to finish:

```
git clone https://github.com/SHIVAAKARTHIK/conveyor-backend.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-admin-web.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-firmware-.git
git clone https://github.com/SHIVAAKARTHIK/conveyor-docs.git
```

Each prints lines such as `Cloning into 'conveyor-backend'...` and finishes in a few seconds. No account or password is asked. Check:

```
dir
```

```
conveyor-admin-web
conveyor-backend
conveyor-docs
conveyor-firmware-
```

5. Start the backend

The backend is the server that stores the accounts, prescriptions and deliveries.

5.1 Open PowerShell in the backend folder

Open `C:\MediDispense\conveyor-backend` in File Explorer, click the address bar, type `powershell`, press Enter (section 2.2). Keep using **this** window for 5.2 to 5.6.

5.2 Create the Python environment

```
python -m venv .venv
```

This makes a private folder called `.venv` that holds this project's Python packages. It takes about 10 seconds and prints nothing.

5.3 Install the backend's packages

```
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

This downloads about 40 packages and takes **1 to 5 minutes**. Lots of text scrolls past. It ends with a line starting `Successfully installed`. A message that a new version of pip is available is harmless.

5.4 Copy the settings file

```
copy .env.example .env
```

This makes the backend's settings file `.env` from the template. There is no message. You do not need to change anything in it. (Check with `dir .env`.)

5.5 Create the database with the demo accounts

```
.\.venv\Scripts\python.exe seed_demo.py
```

```
Added 13 account(s); 0 already existed and were left unchanged.
Database: sqlite:///./dev.db
```

This creates the database file `dev.db` with the admin, doctors, nurses, a technician and the conveyor's login (Part 7). You can run it again safely.

5.6 Start the backend

```
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

```
INFO:     Started server process [1234]
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
```

The window now stays busy with no new prompt. That is correct. Leave it open. Do not close it while you use the system. Use a **new** PowerShell window for anything else.

Keep `--host 0.0.0.0` in the command. Without it, only this computer can reach the backend, and the conveyor never can. If Windows shows a **Windows Security Alert** about Python, tick **Private networks** and click **Allow access**.

5.7 Check it works

Open Chrome or Edge and go to **http://localhost:8000/docs**. You should see a page titled **MediDispense API** listing many endpoints.

5.8 Let the conveyor reach the backend (one time)

The conveyor board calls the backend over WiFi, and the Windows firewall can block that. Do both of these:

1. **Make your WiFi a Private network.** Windows 11: **Settings, Network & internet, Wi-Fi**, click your WiFi's name (properties), and under **Network profile type** choose **Private network**. Windows 10: **Settings, Network & Internet, Wi-Fi**, click your WiFi, and under **Network profile** choose **Private**.
2. **Open PowerShell as administrator** (section 2.7) and paste these **three lines**, pressing Enter after each:

   ```
   $p = @{DisplayName="MediDispense backend"; Direction="Inbound"; Protocol="TCP"}
   $p.LocalPort = 8000; $p.Action = "Allow"; $p.Profile = "Private"
   New-NetFirewallRule @p
   ```

   It prints a short table of settings (Name, DisplayName, Enabled True...). You can close that administrator window afterwards. This only lets **private** networks reach port 8000.

5.9 Stop the computer going to sleep

If the laptop sleeps, the backend stops. **Settings, System, Power** (Power & battery on Windows 11): set "When plugged in, put my device to sleep after" to **Never**, and keep the laptop plugged in while the conveyor is running.

6. Start the web app

6.1 Open a second PowerShell in the web app folder

Do **not** use the backend window. Open `C:\MediDispense\conveyor-admin-web` in File Explorer, click the address bar, type `powershell`, press Enter.

6.2 Install the web app's packages

```
npm install
```

It takes **1 to 3 minutes** and ends with a line like `added 400 packages`. Warnings about "deprecated" packages or "vulnerabilities" are harmless: ignore them, and do **not** run `npm audit fix`.

6.3 Copy the settings file

```
copy .env.example .env.local
```

No message means success. You do not need to edit it: it already points at the backend on this computer (`http://localhost:8000`).

6.4 Start the web app

```
npm run dev
```

```
▲ Next.js 16.x
- Local:  http://localhost:3000
✓ Ready in ... ms
```

As with the backend, this window stays busy. **Leave it open.** You now have two PowerShell windows running: the backend and the web app.

6.5 Open it

In Chrome or Edge go to **http://localhost:3000**. The first load takes up to half a minute because the page is built the first time. You should see the **MediDispense sign-in** page.

7. First login and accounts

These accounts were created by step 5.5. Sign in on the web app's first page with an email and a password. **Log out** with the button at the top right before signing in as someone else.

Demo passwords. They are public, so they are fine on your own computer for a project demo. The web app has no "change password" screen yet: to change one, sign in as admin, open **Staff**, delete the account and create it again with a new password. Do this before anyone outside your team uses the system.

Role	Name	Email	Password
Admin	Default Admin	`admin@medidispense.com`	`admin123`
Nurse	Nurse Priya	`nurse@medidispense.com`	`pass123`
Nurse	Nurse Ramesh	`nurse2@medidispense.com`	`pass123`
Technician	Technician Suresh	`technician@medidispense.com`	`pass123`
Conveyor (machine)	Conveyor Station	`conveyor.station@medidispense.com`	`station123`

Doctors (password `pass123` for all)

Speciality	Doctor	Email
Cardiology	Dr. Arjun Rao	`dr.arjun.rao@medidispense.com`
	Dr. Meera Nair	`dr.meera.nair@medidispense.com`
Gastroenterology	Dr. Kavya Shetty	`dr.kavya.shetty@medidispense.com`
	Dr. Rohit Verma	`dr.rohit.verma@medidispense.com`
Nephrology	Dr. Anita Desai	`dr.anita.desai@medidispense.com`
	Dr. Sanjay Kulkarni	`dr.sanjay.kulkarni@medidispense.com`
Pulmonology	Dr. Farhan Ali	`dr.farhan.ali@medidispense.com`
	Dr. Lakshmi Iyer	`dr.lakshmi.iyer@medidispense.com`

Patients have no preset logins: each patient registers with **Sign up** on the first page and chooses their own email and password. A new doctor you create yourself needs a **department** and must press the green **Available** button before patients can find them.

Role	Sees after signing in
Patient	Home (their deliveries and an "I received it" button), Find a Doctor, My Consultations

Doctor	Consultations: Available/Busy switch, accept requests, write prescriptions
Nurse	Bedside confirmation: assign Bed 1 or Bed 2 to patients, confirm arrived deliveries
Technician	Conveyor: live hardware control, deliveries in progress, alerts; Install firmware
Admin	Dashboard, patients, staff, prescriptions, consultations, alerts, reports; can also control the conveyor

8. Build the conveyor

You can build the hardware at the same time as, or after, the software. **Keep the USB cable and the 12 V adapter unplugged** while you assemble and wire. Wiring while powered once made the board restart and can damage parts.

8.1 Assemble the belt

1. Build the conveyor kit by following **its own instruction manual**, as a plain motorised belt first.
2. Make sure the belt turns smoothly by hand and the rubber belt is centred and not rubbing the frame.
3. Take the kit's **battery holder out of the circuit**. Nothing in this project uses batteries for the belt; the ESP32 controls the motor through the L298N.

8.2 Make the parts for the pushers and chutes

- **Two pusher arms:** glue **three layers** of thin plastic sheet together so the arm is stiff. Each arm should reach about **7.5 cm** from the servo horn. Glue a small **L-shaped paddle** under the tip so its flat face pushes the box.
- **Two servo risers:** fold a hollow box from cardboard for each servo to stand on. The riser's height is right when the paddle hangs a few millimetres above the belt's rail but still meets the box.
- **Two chutes:** small cardboard trays that catch the box when it is pushed off the belt. One goes on each side, as described below.
- **A test box** of about **4 x 3 x 1.5 cm**. It must be narrower than the belt (6.5 cm) with margin.

8.3 Where everything goes

Each bed has its own **pusher line**: one sensor and one servo. The two lines are about 6.5 cm apart along the belt. Look at the belt with the **loading end** on your left.

	Bed 1 (first line, nearer the loading end)	**Bed 2 (second line, further away)**
Servo	Servo 1 on the **near side** of the belt	Servo 2 on the **far side** of the belt
Pushes the box into	The Bed 1 chute on the **far side**	The Bed 2 chute on the **near side**
Sensor	Sensor A, just **before** servo 1's sweep	Sensor B, just **before** servo 2's sweep

- **The two servos must be on opposite sides.** An arm is longer than the gap between the two lines, so if both servos were on the same side the parked arm of one would collide with the other.
- Each sensor faces the belt so that the box passes within a couple of centimetres of its face. A sensor's small **LED lights** when something is in front of it. Fix them firmly so they do not move.
- Put the **button and buzzer** at the loading end, away from the arms' sweep. The ESP32 and the L298N sit nearby, off the belt.
- Both arms rest at **0 degrees** (parked beside the belt, parallel to its edge) and swing to **90 degrees** across the belt. **Do not press the horns onto the servos yet**: do it in section 10.2, when the servos are powered and parked at 0 degrees, so the arms sit in the right position.

8.4 Soldering

Soldering joins a wire to a component permanently. If you have never done it, ask for a quick demonstration. Hold the iron on **both** the pin and the wire for 2 to 3 seconds, feed in a little solder, let it cool for a few seconds without moving, then tug gently to test. Cover bare joints with tape or heat-shrink so they cannot touch each other.

- **IR sensors:** the small plug on the sensor module grips jumper wires badly and gives false readings. **Solder a wire directly onto each of the three pins** (G, V, S) of both sensors.
- **Push button:** solder a wire to each of its two legs.
- **Buzzer:** solder a wire to each of its pins (SIG, VCC, GND). The fourth pin (NC) stays empty.
- Use the wire colours in section 8.5 so you can tell wires apart later.

8.5 Wiring

Match every wire to the table. Push each jumper in **firmly**: loose wires were the most common fault during testing.

Device	Wire	Goes to ESP32 pin
Sensor A (Bed 1)	S, orange	**D4**
Sensor B (Bed 2)	S, orange	**D32**
Servo 1 (Bed 1)	signal, yellow	**D18**
Servo 2 (Bed 2)	signal, yellow	**D19**
Buzzer	SIG, black	**D23**
Push button	one leg (the other leg goes to ground one)	**D15**
L298N ENA (speed)		**D25**
L298N IN1, IN2 (direction)		**D26**, **D27**

The pins are printed on the ESP32 as **D4**, **D32**, and so on (some boards print only the number, such as 4 or 32). The ESP32 has only two ground pins, one 3.3 V pin and one 5 V pin, but many parts need power and ground. So the wires are **joined into four shared rails**, and each rail goes to **one** ESP32 pin. Join the wires of a rail with a small breadboard (one row per rail), screw or lever connectors, or by twisting and soldering them, then cover the joint with tape.

Rail	Goes to ESP32 pin	Wires joined on it
Ground one	one **GND** pin (the one beside VIN)	Both sensors' G (green), the button's other leg, the buzzer's GND (yellow)
3V3	**3V3**	Both sensors' V (brown)
Ground two	the **other GND** pin (beside 3V3)	Both servos' ground (brown), the L298N's GND
VIN (5 V)	**VIN** (not VN)	Both servos' power (red), the buzzer's VCC (white)

Motor side (L298N)	Goes to
The belt motor's two wires	The two **output** screw terminals OUT1 and OUT2. Strip the wire ends so bare copper is clamped fully **under** the screw, tighten, and tug each wire. (Swap them if the belt runs the wrong way.)
12 V adapter +	L298N **12V** terminal
12 V adapter −	L298N **GND** terminal, which is also joined to ground two
ENA jumper (a small plastic cap on two pins marked ENA)	**Remove it**, and wire ENA to D25 instead. Left in place, the speed cannot be controlled.
L298N 5V terminal	Leave empty

> **Two easy mistakes.** (1) Brown is **ground** on the servos but **3.3 V power** on the sensors, so keep those rails apart. A sensor's V wire must end on **3V3**, never on a GND pin: a sensor wired to ground has no power and its LED stays dark. (2) The 12 V adapter powers **only** the L298N and the belt motor. The ESP32 runs from the USB cable.

8.6 Check before switching on

1. No bare wire or joint touches another.
2. The ENA jumper on the L298N is removed.
3. Adapter + is on 12V and adapter − is on GND.
4. Every sensor's V wire is on 3V3 and G is on ground one.
5. The servo arms are **not** attached yet.

9. Put the firmware on the board

The firmware is the program that runs on the ESP32. Plug the board into the computer with the USB cable first.

9.1 About config.h (and copying config.h.example to config.h)

The firmware folder holds a template named `config.h.example`. To build the firmware, a file named `config.h` must exist next to it. It holds the pin numbers and default settings. You normally **do not edit it**: the WiFi and login are entered on the board's own setup page (9.4), not in this file.

- If you use the **flash script** (option A below), it creates `config.h` for you. Nothing to do.
- If you want to do it yourself, copy the file with the command or with File Explorer, exactly as in **section 2.6**. In PowerShell, in `C:\MediDispense\conveyor-firmware-`:

  ```
  copy config.h.example config.h
  ```

- The browser installer (option B) needs neither.

> **Never share or upload your own config.h** if you edit it to contain a WiFi password. The project's `.gitignore` already keeps it out of Git.

9.2 Option A: flash with one command

1. Open PowerShell **in the firmware folder** `C:\MediDispense\conveyor-firmware-` (section 2.2).
2. Make sure the board is plugged in, and close anything else that uses it (the Arduino IDE, a serial monitor).
3. Run:

```
powershell -ExecutionPolicy Bypass -File .\flash.ps1
```

The script does everything by itself, printing a heading for each step: it checks `arduino-cli`, creates `config.h` if needed, installs the ESP32 board support and two libraries (the first time only), finds the board's COM port, compiles, and uploads.

> **The first run takes 5 to 15 minutes** because it downloads the ESP32 board support (several hundred MB). It looks stuck for a while but is working: wait. Later runs take about a minute.

```
==> Compiling the firmware
Sketch uses 1132007 bytes (86%) of program storage space...
==> Uploading to the board ...
Hash of data verified.
Done. The firmware is on the board.
```

Option	When to add it to the command

`-Port COM5`	The script finds more than one serial port, or you want to choose (use your COM number)
`-CompileOnly`	You only want to check that it builds, with no board connected

If the upload hangs at "Connecting...", hold the **BOOT** button on the ESP32 until it starts, then release it. If no board is found, see Part 14.

9.3 Option B: install from the browser (no command line)

1. Make sure the backend and the web app are running (Parts 5 and 6).
2. Open **http://localhost:3000** in **Chrome or Edge** and sign in as the technician or admin (Part 7).
3. Click **Install firmware** in the menu at the top.
4. Plug the board in. Close the Arduino IDE and any serial monitor.
5. Leave **Keep the board's saved WiFi and login** ticked for an update. **Untick** it for a brand-new board.
6. Click **Connect and install**. A popup lists serial ports: choose **CP210x USB to UART Bridge** (or the COM number you wrote down) and click **Connect**.
7. Watch the progress bar. It takes about a minute. When it says **Installed**, the board restarts.

If it stalls at "Connecting", hold the board's **BOOT** button and press install again. If you change the servo angles or belt speed in `config.h` (section 10.3), use option A, because the browser installer's files use the default values.

9.4 The setup page: tell the board your WiFi (once)

A brand-new board does not know your WiFi yet, so it creates its **own** WiFi network and shows a setup form.

1. After flashing, the board's blue **LED blinks fast**. It opens a WiFi network called **MediDispense-Setup** (password `medidispense`). Allow about 30 seconds.
2. On a **phone or second laptop**, open its WiFi list and join **MediDispense-Setup**. Enter the password `medidispense`. Your phone may say "no internet": choose to **stay connected**. If the phone keeps jumping back to your home WiFi or mobile data, turn mobile data off for a minute.
3. Open a browser on that phone and type **http://192.168.4.1** in the address bar (type **http**, not https, and not in a search box). A page titled **MediDispense conveyor setup** appears.
4. Fill in the form:

Field	What to put
WiFi network	Choose your WiFi from the list (2.4 GHz only) and type its password in "WiFi password"
Backend computer name	The name of the computer where you started the backend. On that computer run `hostname` in PowerShell and type exactly what it prints
Backend computer address (backup)	`http://` plus that computer's IP address plus `:8000`. On that computer run `ipconfig` and read "IPv4 Address", for example `http://192.168.1.25:8000`
Conveyor login (email)	`conveyor.station@medidispense.com`
Conveyor login (password)	`station123`

5. Tap **Save and restart**. The page says **Saved**. The board restarts and joins your WiFi. Reconnect your phone to its normal WiFi.

> The setup page **opens again by itself** if the board cannot join its saved WiFi within 30 seconds. To open it **on purpose**, hold the push button for 3 seconds while powering the board on. It only exists on the board's own network, so nobody on your normal WiFi can change the settings. The WiFi password is stored on the board and is never shown again.

9.5 Check that the board is online

On the computer, open Chrome or Edge and go to **http://conveyor.local/status**. You should see a line of settings that includes `"mode":"delivery"` and `"settings":"saved"`. (If `conveyor.local` does not open, see Part 14.) You will also see **Conveyor online** on the technician page in the next part.

10. Test the hardware

10.1 Open the control panel

1. Sign in as the **technician** (Part 7). You land on the **Conveyor** page.
2. Find the box **Live hardware control**. It must say **Conveyor online** and show cards for **Belt**, **Sensor A (Bed 1)**, **Sensor B (Bed 2)** and **WiFi**.

10.2 Check each part, in this order

Do this	You should see or hear
Click **Beep**	The buzzer sounds. If not, check its three wires (section 8.5).
Hold your hand 2 to 3 cm in front of sensor A, then sensor B	The sensor's own LED lights, and its card on the page changes to **Box seen** (red). Take your hand away and it returns to **Clear**.
Click **Push Bed 1**, then **Push Bed 2 with no arms fitted**	Each servo's horn turns and comes back. They always return to **0 degrees**.
Fit the arms now. With the servos parked at 0 degrees, press each arm onto its horn so that it lies **beside** the belt, clear of the box path, and screw it on. Keep your hands away from the arms from now on.	Click **Push Bed 1**: the arm swings across the belt and returns. Do the same for Bed 2.
Click **Start belt**, then **Stop belt**	The belt moves, then stops. If it does not move, check the L298N's power LED and the motor wires (Part 14).

10.3 Adjusting the servo angles and belt speed (only if needed)

The arms swing from 0 to 90 degrees by default. If an arm does not reach the box, or hits something, tune the angles:

1. Close the web app's use of the board, then open PowerShell and run (use your COM number): `arduino-cli monitor -p COM3 -c baudrate=115200`
2. Type `1` to select servo 1 (or `2` for servo 2). Type `.` to raise the angle by 5, `,` to lower it, then `p` to try a push. `v` shows the saved angles. Press **Ctrl+C** to leave.
3. These keys only change the angles **until the board restarts**. To keep them, open `config.h` (right-click it, **Open with**, **Notepad**), change the numbers `SERVO1_PUSH`, `SERVO2_PUSH` (and `SERVO1_REST`, `SERVO2_REST` if needed) and, if you wish, `BELT_SPEED`, save, and flash again with **option A** (9.2).

11. Create and run a delivery job

The key idea. The **doctor's prescription creates the job** automatically. The technician does not create it; the technician loads the box and starts the belt. A **nurse assigns the bed** first, because the bed decides the route: **Bed 1** uses sensor A and servo 1; **Bed 2** uses sensor B and servo 2. Before you begin, the backend, the web app and the conveyor must all be running (Part 12), and the technician page must say **Conveyor online**.

11.1 Patient registers (no bed yet)

1. Open **http://localhost:3000**. If someone is signed in, click **Logout** (top right). On the sign-in page click **Sign up**.
2. Fill in full name, age, gender, optional phone, email and password. Use a real-looking email such as `student1@example.com` and remember the password.
3. Click **Check in**. The home page says "*Waiting for a nurse to assign your bed.*"

11.2 Nurse assigns the bed

1. Click **Logout**. Sign in as a **nurse** (`nurse@medidispense.com`). You land on **Bedside confirmation**.
2. Under **Patients and beds**, the new patient is highlighted at the top. Open its dropdown, choose **Bed 1** or **Bed 2**, and click **Assign**. The page says the patient is now in that bed.

11.3 Patient asks a doctor

1. Logout, then sign in as the patient with the email and password from 11.1.
2. Click **Find a Doctor**, choose a speciality (for example Cardiology), and pick a doctor who shows **Available** in green.
3. Type what is wrong (for example "chest pain since last night") and click **Send request**. The page says "*Request sent*".

11.4 Doctor accepts and prescribes (this creates the job)

1. Logout. Sign in as the **same doctor** (for example `dr.arjun.rao@medidispense.com`).
2. The green **Available** button at the top right must not say **Busy**.
3. Under **Requested**, click **Accept**. The request moves to **Accepted: ready to prescribe**.
4. Click **Prescribe**. Type the **medicine name**, the **dosage** (for example 500mg), choose the form, set the quantity, and add notes if you like. Click **+ Add another medicine** for more.
5. Click **Send prescription**. A new line appears under **Medicine deliveries** with the status `preparing` . **This is the job.**

If you see "*This patient has no bed yet. Ask a nurse to assign one before prescribing*", go back to 11.2.

11.5 Technician starts the belt

1. Logout. Sign in as the **technician**. Under **Live hardware control**, check **Conveyor online**.
2. Within about 4 seconds the board **beeps** once, and the panel says "*A box for Bed N is waiting.*" It also appears under **Deliveries in progress**.
3. **Put the test box on the belt at the loading end, before sensor A.** Keep your hands clear of the servo arms.
4. Press the machine's **push button once**, or click **Dispatch Bed N box** on the page.

What happens next, by itself: the belt starts; the sensor at that bed sees the box and stops the belt (a Bed 2 box rides past sensor A, which is ignored); that bed's servo pushes the box into the chute; the buzzer gives one long beep; the delivery changes to `arrived` . This takes about 5 seconds.

Press only once. A second press during the run is the **emergency stop** and marks the delivery failed. A run also fails if the box does not reach its sensor within **40 seconds**. If nothing happens when you press, no job is waiting: check that the page shows a `preparing` delivery. For the next job nothing needs flashing: when another prescription is waiting, press the button again and it starts that job.

11.6 Nurse confirms at the bedside

1. Logout, then sign in as a **nurse**. Under **Ready to confirm**, find the delivery marked `arrived` .
2. Enter the patient's wristband code (a barcode scanner types it for you). Without a scanner, click **Fill from record (demo)**.

3. Click **Confirm delivery**. A wrong code is rejected with a message. The delivery becomes `confirmed` and the prescription `delivered`.

11.7 Patient sees the result

Sign in as the patient. The home page shows **Delivered**. If the nurse has not confirmed yet, the patient can click **I received it** on an `arrived` delivery.

12. Every time you use it

Start

1. **Power the conveyor:** plug the ESP32's USB into the laptop (or a USB charger) and plug in the 12 V adapter. Wait about 15 seconds.
2. **Start the backend.** Open PowerShell in `C:\MediDispense\conveyor-backend` and run:

   ```
   .\.venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
   ```

3. **Start the web app.** Open a **second** PowerShell in `C:\MediDispense\conveyor-admin-web` and run:

   ```
   npm run dev
   ```

4. Open **http://localhost:3000** in Chrome or Edge. The technician page must say **Conveyor online**.

You do **not** repeat `npm install`, `pip install`, `copy ...` or `seed_demo.py` each time: those were one-time steps. Your data is kept between runs.

Stop

Click in each PowerShell window and press **Ctrl+C**, then close the windows. Unplug the 12 V adapter when you finish. The conveyor only works while the backend is running and the laptop is awake on the same WiFi.

13. WiFi, new computer, updates, reset

New WiFi (no cable or reflashing needed)

1. Switch the conveyor off and on. When it cannot join the old WiFi, **MediDispense-Setup** appears after about 30 seconds. (If the old WiFi still works, hold the push button for 3 seconds while powering on instead.)
2. Join **MediDispense-Setup** (password `medidispense`) and open **http://192.168.4.1** (steps 9.4).
3. Pick the new WiFi, enter its password, and tap **Save and restart**.

The backend now runs on a different computer

Set that computer up with Parts 3 to 6 and 5.8. Then open the conveyor's setup page (as above) and enter that computer's **name** (`hostname`) and **address** (`ipconfig`). If you open the web app from another device, add that address to `CORS_ORIGINS` in the backend's `.env`.

The computer's IP address can change when the router restarts. The board looks the computer up by name every minute, so this is usually fine. If not, update the address on the setup page, or give the computer a fixed IP in the router.

Getting the latest version of the project

Stop the backend and web app (Ctrl+C). In PowerShell, in each of the four folders, run `git pull`. Then: in `conveyor-backend` run `.\.venv\Scripts\python.exe -m pip install -r requirements.txt`; in `conveyor-admin-web` run `npm install`; and flash the firmware again (option A or B in Part 9) if the firmware changed. Then start everything as in Part 12.

Start again with clean data

Stop the backend (Ctrl+C). In the backend folder run `del dev.db`, then `.\.venv\Scripts\python.exe seed_demo.py` again. All patients, prescriptions and deliveries are removed, and the demo accounts come back.

Clear what the board has saved

Plug the board in by USB and run `arduino-cli monitor -p COM3 -c baudrate=115200`. Type `c` to clear the saved WiFi and login; the board restarts and opens its setup page. Press **Ctrl+C** to leave. (Or install the firmware from the browser with "Keep saved WiFi and login" **unticked**.)

14. If something goes wrong

Installing and PowerShell

What you see	What to do
`... is not recognized as the name of a cmdlet` for `git`, `python`, `node`, `npm` or `arduino-cli`	You opened PowerShell before installing it. **Close every PowerShell window and open a new one.** If it still fails, the install did not finish or PATH was not set: repeat that program's step in Part 3.
`winget` is not recognized	Use "Way B" (download and run the installer) for that program.
Python was not found; run without arguments to install from the Microsoft Store	See the yellow box in section 3.2.
running scripts is disabled on this system	Use the command exactly as written, which includes `-ExecutionPolicy Bypass`. You do not need to change any Windows setting.
`git clone` says "Could not resolve host"	No internet. Check your connection and try again.
`pip install` shows red errors or stops	Check the internet, then run the same command again. Make sure you used `.\.venv\Scripts\python.exe -m pip ...` from inside `conveyor-backend`.

Backend and web app

What you see	What to do
Backend: "error while attempting to bind ... port 8000" (address already in use)	A backend is already running in another window. Close that window (or Ctrl+C), then start again.
Web app: "Port 3000 is in use"	Another web app window is open. Close it, or the page will be on another port that the message shows.
Sign-in says "Invalid credentials"	Did you run `seed_demo.py` (step 5.5)? Check the email and password in Part 7, with no extra spaces. Is the backend window still running?
The web page cannot reach the backend, or lists are empty	Is the backend window open and showing "Uvicorn running"? Does http://localhost:8000/docs open? Did you create `.env.local` (step 6.3)?
The first page takes a long time	Normal the first time (up to 30 seconds). It is fast afterwards.
Doctor not listed for the patient	The doctor must be **Available** (green button) and have a department. The demo doctors already do.
Doctor cannot prescribe	The patient has no bed yet. A nurse must assign one (11.2).

The conveyor board and WiFi

What you see	What to do
The flash script says "No board found"	Is the board plugged in with a **data** USB cable? Does Device Manager show a COM port (3.6)? Install the driver or try another cable, then run it again. Or add `-Port COM5` with your number.
Upload hangs at "Connecting..."	Hold the **BOOT** button on the ESP32 until the upload starts, then release it.
Upload fails with "port is busy" or "access denied"	Something else holds the port: close the Arduino IDE, any serial monitor, and the browser installer tab. Unplug and replug the board.
The first flash seems frozen for minutes	It is downloading the ESP32 support (several hundred MB). Wait up to 15 minutes.
Browser installer: "No board was chosen"	The popup was closed without picking a port. Press the button again and choose the COM port.
Browser installer cannot connect	Use Chrome or Edge. Close other programs using the board. Hold **BOOT** and try again. Check the cable and driver.
I cannot see **MediDispense-Setup** on my phone	Wait 30 seconds after power-on and refresh the WiFi list. Hold the push button 3 seconds while powering on to force it. The blue LED should blink fast.
The setup page will not open	Use **http://192.168.4.1** (not https). Make sure the phone is still joined to **MediDispense-Setup**, not back on another WiFi.
"Conveyor not reachable" on the web page	Is the board powered and on the same WiFi as this computer? The page shows the setup steps. If `conveyor.local` does not work (a VPN or some routers), find the board's IP address and type it under **Conveyor address** on the panel.
The board never picks up a job	Backend started with `--host 0.0.0.0`? Firewall rule added (5.8)? Same WiFi? The conveyor login and backend name saved on its setup page correct? Re-open the setup page (hold the button 3 seconds at power-on) and check them.
The board restarts on its own	A loose USB cable, a loose wire, or a wire plugged in while powered. Unplug, re-seat everything, plug in again.

Hardware

What you see	What to do
Belt does not move	Is the L298N's power LED lit and the 12 V adapter plugged in? Are the two motor wires tight under the OUT1 and OUT2 screws (tug them)? Are the ENA, IN1 and IN2 wires on D25, D26, D27, and is the ENA jumper removed?
A sensor never reacts	Its LED should light when you pass a hand in front. Dark LED: check V on **3V3** (not a GND pin) and G on ground one. LED works but the page does not change: check the S wire on D4 (sensor A) or D32 (sensor B).
The box never stops at the sensor	The box must start on the belt **before** the sensor and pass close to its face. Check the sensor LED lights as the box passes.
A servo does not move	Signal on D18 (servo 1) or D19 (servo 2); red on VIN; brown on ground two. Test with **Push Bed 1** and **Push Bed 2**. Servos are weak if the arm rubs: free the arm.

No buzzer sound	White VCC to the VIN rail, yellow GND to ground one, black SIG to D23. Test with **Beep**.
The box is pushed too weakly or misses the chute	Adjust the angles (10.3) or the arm length and chute position.

Jobs

What you see	What to do
Button or Dispatch does nothing	No job is waiting. Check that a prescription exists and a `preparing` delivery is listed.
The run ended as "failed"	A second button press (emergency stop), or 40 seconds passed without the box reaching its sensor.
A delivery is stuck in progress	On the technician page click **Mark failed** next to it.

15. Before real use, quick reference

Before anyone else uses it

- Change the demo passwords (Part 7): delete and recreate each account on the admin **Staff** page.
- Set a long random `JWT_SECRET_KEY` in the backend's `.env` and restart the backend.
- The conveyor's local control page uses plain HTTP and has no password. Keep it on a private network.
- The setup WiFi password (`medidispense`) is public. Do the setup yourself, and do not leave the board in setup mode.
- This is a model-scale prototype for two beds and one box at a time. It is not a clinical device and has not been validated for real medicines.

Addresses, ports and files

What	Where
Web app	http://localhost:3000 (port 3000)
Backend and its API page	http://localhost:8000 and http://localhost:8000/docs (port 8000)
Conveyor status	http://conveyor.local/status (the board, port 80)
Conveyor setup page	WiFi **MediDispense-Setup** (password `medidispense`), then http://192.168.4.1
The database	`conveyor-backend\dev.db`
Backend settings	`conveyor-backend\.env` (copied from `.env.example`)
Web app settings	`conveyor-admin-web\.env.local` (copied from `.env.example`)
Firmware settings	`conveyor-firmware-\config.h` (copied from `config.h.example`)

Words used in this guide

Terminal / PowerShell	A window where you type commands.
Repository (repo)	A project folder stored on GitHub, downloaded with `git clone`.
Backend	The server program that stores the data and talks to the web app and the conveyor.
Firmware	The program running on the ESP32 board.

Flash	To copy the firmware onto the board.
COM port	The name Windows gives the USB connection to the board, such as COM3.
Sensor A / B	The infrared sensors for Bed 1 and Bed 2.
Servo	A small motor that swings an arm to push the box.
Dispatch	Start the waiting delivery (the push button does the same).